

Adithya Palaniappan Ramesh

925-738-8393 | San Francisco Bay Area | adithya.p.ramesh@gmail.com | [linkedin.com/in/adithyapramesh](https://www.linkedin.com/in/adithyapramesh) | github.com/AdithyaPR18

EDUCATION

Purdue University

Bachelor of Science in Artificial Intelligence

West Lafayette, IN

December 2027

Coursework: OOP, DSA (Java), Artificial Intelligence (Python), Competitive Programming (C++), ML, Databases

TECHNICAL SKILLS

Programming Languages: Python, Java, C++, C, JavaScript/TypeScript, Swift

Frameworks/Libraries: Flask, FastAPI, React, Spring Boot, Django, OpenCV, Pandas, PyTorch, SciKit-Learn, TensorFlow

Tools & Cloud: AWS (EC2, Lambda, S3), Docker, Kubernetes, Hugging Face, ONNX, Git, SQL, CI/CD, Kafka

Databases: PostgreSQL, MySQL, MongoDB, DynamoDB, Redis, Snowflake

EXPERIENCE

IBM

Software Developer Intern · *watsonx.data*

May 2026 – Aug 2026

San Jose, CA

Akina Labs | Purdue Stack

Software Engineering Intern

Jan 2025 - May 2025

West Lafayette, IN

- Optimized edge AI performance by implementing custom **C++ bindings** on a **YOLO-v8** model, reducing **inference latency** by 3x compared to standard Python implementations.
- Automated microscopy workflows through **real time object detection** and **OpenCV** processing, replacing manual labor with high precision edge hardware inference, increasing data analysis speed by **85%**
- Designed a high throughput data ingestion layer in **C++** to buffer and synchronize high resolution camera feeds, preventing frame loss and ensuring consistent **60 FPS** input for the inference pipeline

Syngenta | Purdue Data Mine

Software Engineering Intern

August 2025 – Dec 2025

West Lafayette, IN

- Built a serverless ETL pipeline using **AWS Lambda** to ingest and normalize **Sentinel 2** satellite imagery, reducing data processing time by **30%** and improving reliability of downstream geospatial models
- Developed scalable data modules integrated with internal geospatial pipelines to automate the processing of **multi-terabyte environmental datasets** for predictive modeling, enabling real time analysis for agricultural strategy teams.
- Optimized model inference by exporting PyTorch models to **ONNX Runtime** and applying graph level optimizations, improving inference latency by **40%** in production workflows

ECSite Inc.

Software Engineering Intern

June 2025 – Aug 2025

San Francisco Bay Area, CA

- Built a modular state management layer for large-scale network graph data, reducing client-side memory usage by **35%** and enabling smooth rendering of **5,000+ nodes**
- Implemented rigorous data validation via a **recursive graph traversal algorithm**, ensuring the integrity of complex connectivity logic before deployment to field teams.
- Engineered simulation features within a **3D topology tool** to model and visualize network configurations, reducing iteration time for engineers by **70%**

PROJECTS

Distributed Key Value Store with Leader Election | *C++, TCP, Multithreading*

Dec 2025 - Feb 2026

- Built a **multithreaded TCP-based key-value store** using a custom thread pool and synchronized shared state, supporting concurrent client requests with persistent **write-ahead logging** for crash recovery
- Implemented **leader-follower log replication** with **quorum-based commit** semantics and term-based voting, ensuring **strong consistency** and automatic failover under node crashes
- Designed heartbeat-driven **leader election** and failure detection to prevent **split-brain** scenarios, and benchmarked **throughput and latency** under concurrent load to evaluate safety-availability tradeoffs

Lightweight Vector Database with HNSW Indexing | *Python, FastAPI, C++*

Jan 2026 – Mar 2026

- Engineered a vector database from scratch implementing **HNSW (Hierarchical Navigable Small World)** indexing with configurable M and ef parameters, achieving **sub-millisecond** approximate nearest neighbor search across **1M+ vectors**
- Built a persistent embedding ingestion pipeline integrating **sentence-transformers** for encoding, with **write-ahead logging** for crash recovery and index serialization for cold start performance
- Designed a **FastAPI** query layer supporting top-k similarity search with **recall@k benchmarking**, demonstrating **94% recall** at **10x the throughput** of brute-force cosine similarity